

Taller Express y otras cosas



Nombre: Gabriela Ruelas Gaytán

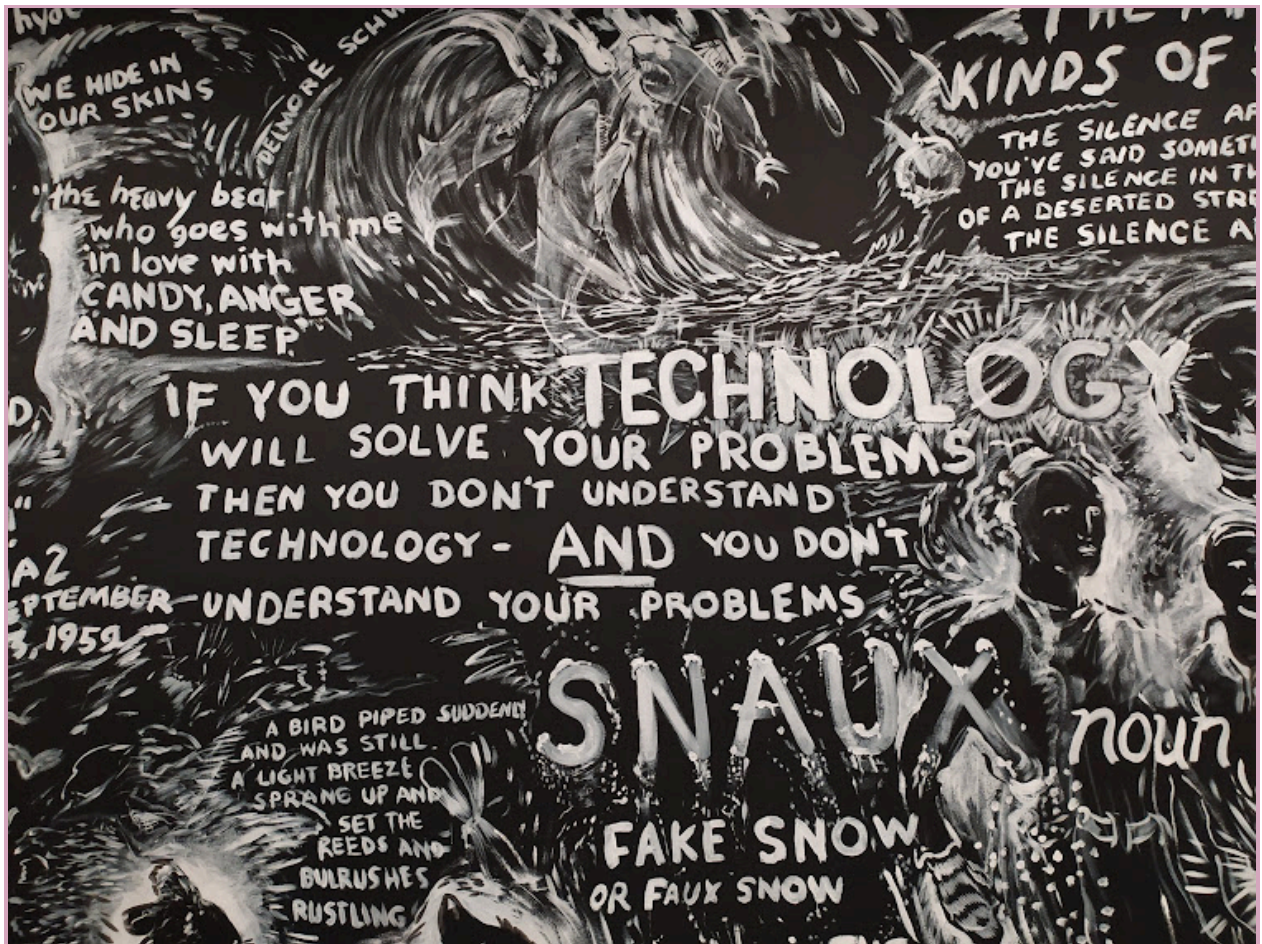
Matrícula: A01640880

¿Quién eres?:

El objetivo de la actividad es explorar y conocer más sobre el desarrollo web del lado del backend.

Al final de la actividad deberás hacer un PDF con este documento. El PDF deberá tener un enlace en [miarchivo.html](#)

- **Ejercicio -1: Una pregunta**



¿Qué piensas de la frase? ¿Qué problema crees que podrías resolver con tecnología? Creo que la tecnología no puede reemplazar el pensamiento crítico ni la comprensión detallada de los problemas. Creo que el poder de la tecnología es que es una herramienta que con un buen uso puede ayudarte a lograr grandes cosas.

- **Ejercicio 0: El pasado**

En las semanas previas vimos la forma de consumir un API desde el cliente (el navegador). Ya consumiste el **API desde el servidor** usando javascript y nodejs.

Escribe aquí abajo el enlace al archivo (con esa solución) en tu repositorio:

<https://github.com/GabyRuelas/GabyRuelas.github.io/blob/main/servidor.js>

¿Cuál fue el principal problema para resolver este ejercicio? ¿Cuál fue el problema que tuvo una de las personas en tu equipo? Explica cómo resolvió esa persona ese problema.

El problema principal que tuve fue entender que no solo el navegador puede hacer solicitudes a una API, también el servidor. Fue confuso saber dónde poner el código y dónde mostrar el resultado.

Uno de mis compañeros tuvo problemas porque el servidor no se ejecutaba correctamente. En la terminal se dió cuenta que le faltaba instalar uno de los módulos. Después de instalarlo, el programa sí funcionó.

• Ejercicio 1: Simplicidad

¿ Recuerdas cómo **instalar un módulo** en node.js? Escribe la instrucción que se usa: `npm install <nombre_del_módulo>`

• Ejercicio 2: Express es

Investiga qué es express (en el contexto de desarrollo web en nodejs). Explica a un **niñ@** qué es express, cómo se usa, para qué sirve. Escribe ese texto aquí. Incluye también **las referencias** de tu investigación.

Express es una herramienta para [Node.js](https://nodejs.org/) que ayuda a crear páginas web, servidores y aplicaciones de forma fácil y rápida.

<https://expressjs.com/#:~:text=Express%20es%20un%20framework%20minimalista,para%20aplicaciones%20web%20y%20m%C3%B3viles>.

Dile a una persona de la clase (que no esté en tu mesa) que te explique qué es express y qué ventajas te podría dar su uso. Graba el audio de la explicación.

<https://www.image2url.com/r2/default/audio/1778895370749-d8fc6433-58aa-4c5d-9786-0663d6137ba4.mp3>

¿Crees que realizó una buena explicación? ¿Por qué?

Sí porque usó palabras sencillas y explicó la función principal de Express que es crear servidores y rutas de manera más fácil y rápida.

• Ejercicio 3: Desde un libro

Investiga una definición en un libro (digital o físico) **de una API**. Incluye la referencia del libro.

API significa Application Programming Interface, es decir, es una forma en que dos sistemas hablen entre sí sin que tú te des cuenta de lo que está pasando dentro.

(S/f). Com.mx. Recuperado el 12 de mayo de 2026, de

https://www.google.com.mx/books/edition/Explotando_La_Inteligencia_Artificial/Y7NaEQAAQBAJ?hl=es-419&gbpv=1&dq=qu%C3%A9+es+un+api&pg=PA301&printsec=frontcover

Escribe un ejemplo de un endpoint de tu proyecto.

```
function mostrarContacto(req, res) {  
  fs.readFile('contacto.html', 'utf8', (error, data) => {  
    if (error) {  
      res.writeHead(500, { 'Content-Type': 'text/plain' });  
      res.end('No se pudo cargar contacto.html');  
      return;  
    }  
  })  
}
```

• Ejercicio 4: Instala express

Escribe la instrucción que se usa para **instalar el módulo de express**:

npm install express

• Ejercicio 5: Uso de express

Ejecuta y explica qué hace el siguiente código:

```
import express from 'express';  
  
const app = express();
```

```
app.listen(1984, () => {  
  console.log('Up and up');  
});
```

Primero importamos Express para que nuestro código pueda funcionar. Después se crea una aplicación utilizando Express(). Luego necesitas que el servidor escuche en el puerto 1984. Y finalmente, cuando el servidor ya está funcionando, aparece el mensaje “Up and up” en la terminal.

• Ejercicio 6: Uso de express++

Ejecuta y explica qué hace el siguiente código (agregado al ejercicio 5):

```
app.get('/bienvenida', (req, res) => {  
  res.send('Esto no es una página html');  
});  
  
app.get('/otraBienvenida', (req, res) => {  
  res.sendFile('bienvenida.html');  
});
```

En caso de que alguno de estos te marque error, soluciona el problema y explica la forma de solucionarlo:

Aquí se agregan dos rutas al servidor. Cuando el usuario entra a BIENVENIDA, el servidor envía un mensaje simple. Cuando el usuario entra por OTRABIENVENIDA, el servidor ahora lo que busca es mandar el archivo BIENVENIDA.HTML. Me marcó que sendFile no tenía la ruta completa del archivo y le agregué el path.resolve().

• Ejercicio 7: Recuerdo de imagen

Es posible si pusiste una imagen en el servidor.js no se podía ver. Un servidor entrega respuestas a solicitudes realizadas por un cliente. Cuando el cliente solicita una imagen (u otro tipo de archivo) el servidor tiene que saber responder.

Express nos propone crear un directorio para los archivos que queremos usar, por ejemplo imágenes o archivos de hojas de estilo.

Investiga qué es express.static.

Escribe tu investigación

Express.static() es una herramienta que permite mostrar archivos como imágenes, páginas html, CSS o JS directamente en tu página web. Solo necesitas indicarle la carpeta en donde están guardados los archivos y Express se encarga de mostrarlos.

Express express.Static() function. (2020, julio 3). GeeksforGeeks.

<https://www.geeksforgeeks.org/web-tech/express-js-express-static-function/>

• Ejercicio 8: Canción

Pregunta a una persona de otro equipo su canción favorita y sus razones.
Here Comes the Sun (The Beatles) porque alegra su día.

¿Ya la conocías? ¿Te gusta? ¿Por qué?

Sí, ya la conocía. Concuerdo con mi compañera. La canción transmite alegría y positivismo. Siento que es una canción que en cualquier momento puede mejorar el ánimo.

• Ejercicio 9: Transformación

Usa express para transformar todo el código de servidor.js (el que tenía lo de opinión), nombra el archivo ahora servidor express.js

Haz comentarios de qué te parece *express* en el mismo código.

Haz que se vea una foto de un árbol de tu universidad con el nombre científico del árbol en una de tus páginas html.

Escribe aquí abajo el enlace al archivo (servidor_express.js) en tu repositorio:

https://github.com/GabyRuelas/GabyRuelas.github.io/blob/main/servidor_express.js

• Ejercicio 10: Verbos

En el contexto de desarrollo web, explica los siguientes verbos GET, POST, PUT, DELETE

GET: pide información al servidor

POST: envía información nueva al servidor

PUT: actualiza información que ya existe

DELETE: borra información

Explica los siguientes códigos, en el contexto de desarrollo web:

1xx - respuestas informativas

2xx - solicitud exitosa

3xx - redirecciones

4xx - error del cliente

5xx - error del servidor

Explica con un ejemplo (relacionado con tu proyecto/reto) cómo se atendería una solicitud de cada uno de esos verbos usando express. Incluye al menos una respuesta 404, 201 y 200.

200 - la solicitud si funcionó

```
app.get('/recomendaciones', (req, res) => {  
  res.status(200).send('Estas son las recomendaciones disponibles para pagar con Kueski');  
});
```

201 - crear un nuevo registro

```
app.post('/interaccion', (req, res) => {  
  res.status(201).send('Interacción del usuario guardada correctamente');  
});
```

200 - actualización correcta

```
app.delete('/widget', (req, res) => {  
  res.status(200).send('Widget cerrado por el usuario');  
});
```

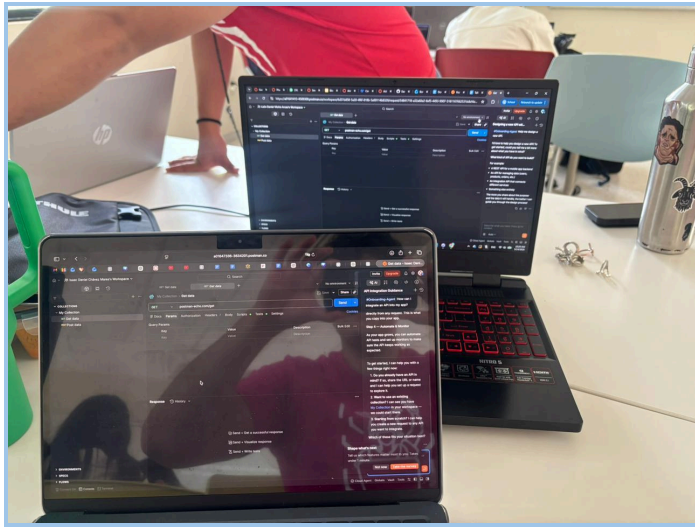
404 - no se encontró un sitio compatible

```
app.get('/sitio-no-compatible', (req, res) => {  
  res.status(404).send('Este sitio no es compatible con Kueski');  
});
```

• Ejercicio 11: Postman

En equipo con tu mesa, instalen postman en cada una de sus computadoras. Tomen una misma foto en la que aparezcan todas las pantallas con postman abierto.

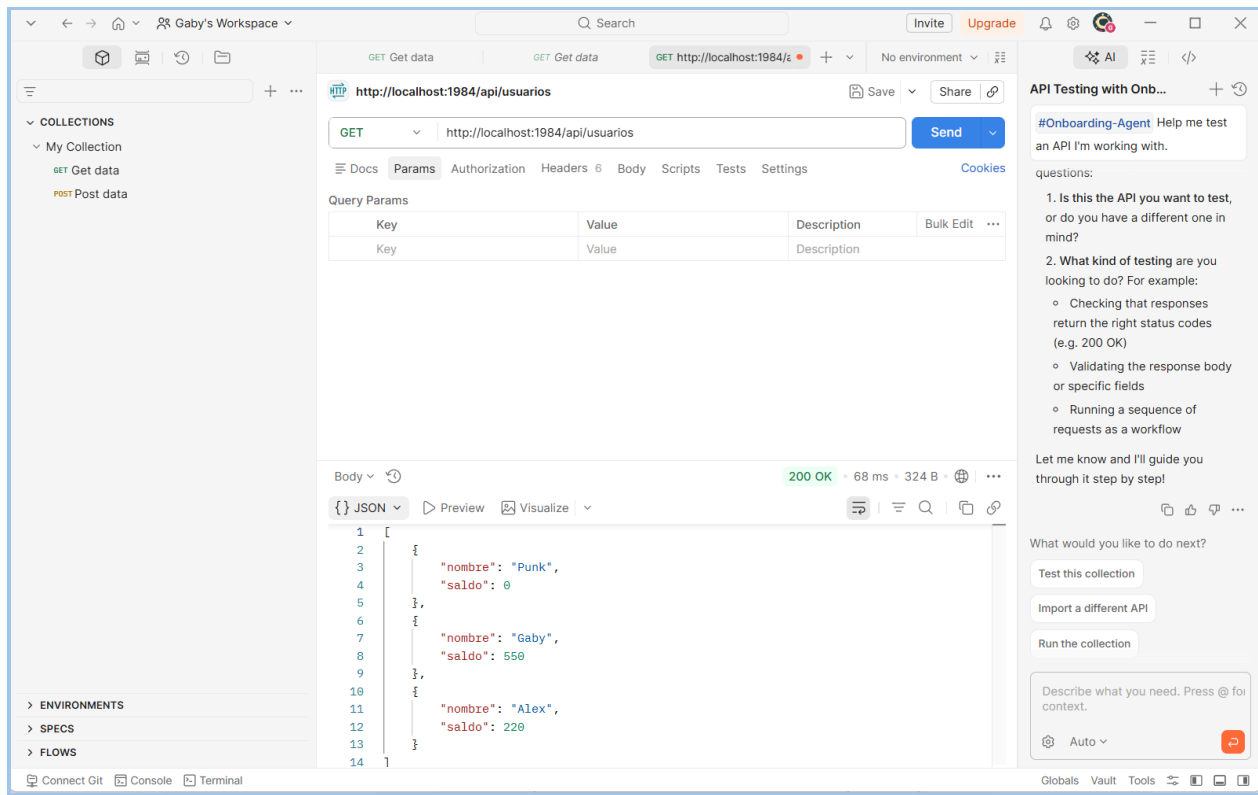
Foto aquí:



Explica para qué sirve Postman

Sirve para probar rutas de un servidor sin la necesidad de usar una página web.

Prueba lo que hiciste en el ejercicio 9 con Postman. Pon una captura de pantalla usando un GET.



• Ejercicio 12: Parámetros

Inventa un ejercicio (que esté relacionado con tu reto) para que uses parámetros en las rutas (por ejemplo /getBancos /postBancos)

La ruta recibe el nombre de una tienda en línea y responde si se está revisando su compatibilidad con Kueski.

Soluciona ese ejercicio

Revisar si amazon es compatible con Kueski

Escribe aquí abajo el enlace al archivo en tu repositorio:

<https://github.com/GabyRuelas/GabyRuelas.github.io/blob/main/ejercicio12.js>

Compara ese ejercicio con el de otra persona de la clase. ¿Qué ejercicio ayuda mejor a comprender el uso de parámetros y por qué?

Me gustó más mi ejemplo porque además de que al hacerlo me dió un poco más de entendimiento sobre el tema, y aparte ayuda a comprender mejor los parámetros

porque el valor escrito en la URL cambia la respuesta del servidor según la tienda con la que estamos trabajando.

• Ejercicio 13: npx

Investiga con alguien más lo siguiente: `npx express-generator`. Importante haz un directorio diferente si quieres ejecutar el comando.

Escribe el nombre de ese alguien más:
Samantha

Explica lo que genera y hace ese comando.

Crea automáticamente la estructura básica de un proyecto con express. Es para tener archivos ya preparados y no tener que empezar de cero.

Explica de manera básica y conceptualmente qué archivos se generaron.

Genera archivos como `app.js` o carpetas como `routes`, que guarda las rutas del servidor, la carpeta `views` que guarda las páginas o plantillas y la carpeta `public` que guarda archivos estáticos como CSS o JS.

• Ejercicio 14: Explicar

Explica a una niña o un niño de 10 años para qué sirven frameworks como Angular, React, Vue, o Svelte.

Los frameworks como Angular, React, Vue o Svelte son herramientas que nos ayudan a crear páginas web más fácil. Hacer una página web es como construir una casa. Sin frameworks, tendrías que hacer todo tú solo y hacerlo desde cero. Con frameworks, ya tienes las piezas listas para usarlas cuando tú quieras. Ya tienes las puertas y las ventanas y las paredes. Esto hace que construir una casa (o crear una página web) sea mucho más fácil y rápido.

• CÓDIGO (con la base de datos del proyecto)

```
const mysql = require('mysql2/promise');

const config = {
  host: 'mysql-2b928616-tec-36a7.c.aivencloud.com',
  port: 27187,
  user: 'avnadmin',
```

```

password: 'AVNS_M7OHtZrjmMMgDnFb8pF',
database: 'defaultdb'
};

async function main() {
  let connection;

  try {
    connection = await mysql.createConnection(config);
    console.log('✅ Conectado a la base de datos');

    // TABLA USUARIOS
    await connection.execute(`
      CREATE TABLE IF NOT EXISTS usuarios (
        id_usuario INT PRIMARY KEY AUTO_INCREMENT,
        nombre VARCHAR(100) NOT NULL,
        edad INT,
        tipo_usuario VARCHAR(100),
        frecuencia_uso VARCHAR(50)
      )
    `);

    // TABLA SITIOS WEB
    await connection.execute(`
      CREATE TABLE IF NOT EXISTS sitios_web (
        id_sitio INT PRIMARY KEY AUTO_INCREMENT,
        nombre VARCHAR(100) NOT NULL,
        tipo VARCHAR(100),
        compatible_kueski BOOLEAN
      )
    `);

    // TABLA WIDGET
    await connection.execute(`
      CREATE TABLE IF NOT EXISTS widget (
        id_widget INT PRIMARY KEY AUTO_INCREMENT,
        nombre VARCHAR(100),
        estado VARCHAR(50),
        tiempo_respuesta FLOAT
      )
    `);
  }
}

```

```
`);
```

```
// TABLA INTERACCIONES
```

```
await connection.execute(`
```

```
CREATE TABLE IF NOT EXISTS interacciones (  
    id_interaccion INT PRIMARY KEY AUTO_INCREMENT,  
    tipo VARCHAR(100),  
    fecha DATETIME,  
    id_usuario INT,  
    id_widget INT,  
    FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario),  
    FOREIGN KEY (id_widget) REFERENCES widget(id_widget)  
)
```

```
`);
```

```
// TABLA RECOMENDACIONES
```

```
await connection.execute(`
```

```
CREATE TABLE IF NOT EXISTS recomendaciones (  
    id_recomendacion INT PRIMARY KEY AUTO_INCREMENT,  
    mensaje VARCHAR(255),  
    tipo VARCHAR(100),  
    id_usuario INT,  
    id_sitio INT,  
    FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario),  
    FOREIGN KEY (id_sitio) REFERENCES sitios_web(id_sitio)  
)
```

```
`);
```

```
// TABLA SCORE COACH
```

```
await connection.execute(`
```

```
CREATE TABLE IF NOT EXISTS score_coach (  
    id_score INT PRIMARY KEY AUTO_INCREMENT,  
    puntos INT,  
    nivel VARCHAR(50),  
    progreso FLOAT,  
    id_usuario INT,  
    FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)  
)
```

```
`);
```

```

console.log('✅ Tablas creadas correctamente');

// DATOS DE PRUEBA
await connection.execute(`
    INSERT INTO usuarios (nombre, edad, tipo_usuario, frecuencia_uso)
    SELECT 'Carlos', 25, 'Comprador frecuente', 'Alta'
    WHERE NOT EXISTS (SELECT 1 FROM usuarios WHERE nombre = 'Carlos')
`);

await connection.execute(`
    INSERT INTO usuarios (nombre, edad, tipo_usuario, frecuencia_uso)
    SELECT 'Ana', 50, 'Usuario ocasional', 'Baja'
    WHERE NOT EXISTS (SELECT 1 FROM usuarios WHERE nombre = 'Ana')
`);

await connection.execute(`
    INSERT INTO sitios_web (nombre, tipo, compatible_kueski)
    SELECT 'Amazon', 'E-commerce', true
    WHERE NOT EXISTS (SELECT 1 FROM sitios_web WHERE nombre = 'Amazon')
`);

await connection.execute(`
    INSERT INTO sitios_web (nombre, tipo, compatible_kueski)
    SELECT 'Liverpool', 'E-commerce', true
    WHERE NOT EXISTS (SELECT 1 FROM sitios_web WHERE nombre =
'Liverpool')
`);

await connection.execute(`
    INSERT INTO widget (nombre, estado, tiempo_respuesta)
    SELECT 'Kueski Smart Widget', 'Activo', 1.5
    WHERE NOT EXISTS (SELECT 1 FROM widget WHERE nombre = 'Kueski Smart
Widget')
`);

await connection.execute(`
    INSERT INTO score_coach (puntos, nivel, progreso, id_usuario)
    SELECT 450, 'Bronce', 45.0, 1
    WHERE NOT EXISTS (SELECT 1 FROM score_coach WHERE id_usuario = 1)
`);

```



```
console.log('✅ Datos de prueba insertados');

const [usuarios] = await connection.execute('SELECT * FROM usuarios');
console.log('\n📋 Usuarios:');
console.table(usuarios);

const [sitios] = await connection.execute('SELECT * FROM sitios_web');
console.log('\n🌐 Sitios web:');
console.table(sitios);

} catch (error) {
  console.error('❌ Error:', error.message);
} finally {
  if (connection) {
    await connection.end();
    console.log('🔌 Conexión cerrada');
  }
}

main();
```